ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Simon Gabriel GELBART**
- **Chinese Name:**
- **Email: t15920143@ntu.edu.tw**
- **Student ID: t15920143**
- **University: NTU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer: 1) 594 documents for the category sci.med

2) The main reason for isolating specific categories, when applying frequent pattern mining, is to remove the noise created when we mix completely different subjects together. Words that appear across all subjects (like "email", "thanks", "best") will show up as patterns, while actual words from the subject are lost because of this noise. By doing that you can find real patterns related to your subject and it will also be faster to compute because you have less words.

AI used for this answer: No

If yes, how:

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: By doing that, we are throwing away 40% of our dataset. So, we have a huge data loss, which directly impacts our analysis afterwards, even though the raw text is still completely useful. A good alternative to this problem is to train a classification model on the remaining 60% of the data and predict the category_name for the missing ones.

AI used for this answer: No

If yes, how:

External references (links):

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer: 1) No, it is not matching perfectly. Even if the sampled distribution looks visually very close to the original, taking a random sampling introduce small statistical variations, meaning the exact proportions are slightly different from the original.

2) The main risk is underrepresenting or completely missing rare classes in the sample.

AI used for this answer: No

If yes, how:

External references (links):

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1) The total number of features increases from 35 482 with stop_words = 'english' to 35 788 with stop_words = None, it makes sense because common English words are not filtered anymore.
2) The sparsity will decrease, because stopwords appear in almost every document and fill columns with non-zero values.

AI used for this answer: No

If yes, how:

External references (links):

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

1) The total vocabulary size decreases (drastically) as the variance threshold increases from 0.0 to 0.05. At a threshold of 0.0, the full vocabulary of 35 482 terms is kept. At 0.01, only 5 089 features are kept and finally at 0.05 we only keep 1 491 features.

2) The type of words having a very low variance are the rare words that appear only once or twice in the documents. It can also be mistake that occur only once. For example, words like “zumder” or “zur” have a very low variance (0.0004).

3) Near-zero variance features do not provide any predictive information and are completely useless for training a model. Removing them reduces the dimensionality but does not change anything to the accuracy of the model.

Threshold	**Number of terms kept (keep == True)**
0.0	35482 kept (0 removed)
0.01	5089 kept (30393 removed)
0.05	1491 kept (33991 removed)

AI used for this answer: No

If yes, how:

External references (links):

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1) The top words are a mix of domain terminology and generic communication words. In Pearson, generic email/forum terms like 'thanks', 'writes', and 'looking' dominate the top ranks, with domain words like 'vga' and 'video' appearing lower down. In contrast, Spearman does a much better job ranking core domain-specific terminology at the very top, such as 'graphics', 'image', '3d', or 'files'.

2) The Pearson correlation measures the strength and direction of a linear relationship between continuous variables. Mathematically, it evaluates how closely the raw data points follow a straight-line equation ($y = ax + b$). It assumes the data is normally distributed and calculates the covariance of raw values normalized by their standard deviations.
The Spearman correlation measures the degree of a monotonic relationship between variables on an ordinal or ranked scale. It is non-parametric and makes no assumptions about data normality. Mathematically, it operates on the differences between the assigned ranks rather than raw numerical counts.

3) This happens when a word appears consistently in low frequencies across documents without huge frequency spikes in any single text. Because Spearman only looks at ranks, a constant presence makes a high score. Pearson, however, requires a linear fit and is dragged down by non-normal, sparse counts. For example, terms like 'graphics', 'image', or '3d' rank high in Spearman due to constant presence but drop in Pearson because they do not have a huge frequency spike in a text.

AI used for this answer: Yes

If yes, how: For question 2, brainstorming and explanation

External references (links):

https://www.statisticssolutions.com/free-resources/directory-of-statistical-analyses/correlation-pearson-kendall-spearman/

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

Decreasing minSup from 9 to 3 caused a large increase in the number of discovered patterns, jumping from 710 to 9 547. A lower threshold is less strict, so it captures many more rare word combinations. This also made the runtime increase from 0.34s to 3.30s because the algorithm has to explore a much larger search space.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

In Top-K, k sets a fixed number of patterns to return, for example exactly the top 100 or 500 most frequent, regardless of their actual support count. In contrast, minSup sets a minimum frequency threshold, which means we don't know how many patterns will be found in advance and it could return zero or thousands of patterns depending on the data.

Answer (3) — Maximal vs standard frequent patterns:

Maximal frequent patterns are more concise because they eliminate redundancy. Instead of returning all sub-patterns that is already included in a larger frequent pattern, an algorithm like MaxFPGrowth only keeps the largest unextendable itemset. For example, if {A, B, C} is frequent, a standard algorithm will return all combinations ({A}, {B}, {A, B}, {C} ...), while maximal mining only returns {A, B, C}.

AI used for this answer: Yes

If yes, how: Rephrase my answer, make sure my answers make sense and helping with the example of the question 3.

External references (links):

https://fr.scribd.com/document/964974334/Closed-Patterns-and-Maximal-Patterns

---

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1) A scatter plot can only display 2 or 3 geometric spatial axes whereas the X_counts matrix represents a high-dimensional space with a lot of dimensions. Projecting data with more than 3 dimensions directly into a 2D or 3D coordinate system is mathematically impossible without dimensionality reduction.
2) We have to use dimensionality reduction algorithms like PCA for example (or t-SNE and UMAP).

AI used for this answer: No

If yes, how:

External references (links):

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1) UMAP produces distinct, separated cluster regions for the categories. We can see an isolation of alt.atheism and a grouping of other topics semantically. In contrast, we can see that PCA fails to separate the classes and to capture any structure, as almost all points seem to be flattened near the origin due to outliers. Finally, we can see that t-SNE shows heavy overlap around the center, the data points are turning into a single dense cloud without distinct boundaries.
2) As noted in the notebook, PCA is a linear technique that finds directions of maximum variance. While it works well for Gaussian-distributed data, it is less effective with non-linear structures and gets sensitive to extreme outliers in high-dimensional text data. In contrast, t-SNE and UMAP are non-linear techniques

that focus on local neighborhoods, making them much better at uncovering complex document groupings.

3) T-SNE mostly uses local neighborhood similarities rather than global geometry. That's why the distance between clusters on the 2D plot doesn't represent real distances in the original high-dimensional space.

AI used for this answer: No

If yes, how:

External references (links):

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1) 4 binary columns are created with one column per class to represent the 4 categories.

2) If we give raw integers to an ML algorithm, it will assume that there is an order, for example, 3 being greater than 1. It might also interpret the values as a metric distance, assuming for example that 3 is closer to 2 than to 0.

3) Instead of keeping categories as integers, one-hot encoding converts each category into its own binary feature, represented by a vector of zeros with a single 1. By doing this, we make all classes linearly independent, ensuring the model treats every category independently without any fake ranking or distance bias.

AI used for this answer: No

If yes, how:

External references (links): https://developers.google.com/machine-learning/crash-course/categorical-data/one-hot-encoding?hl=fr and also one of my home university's course.
